ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT II

Searching and Game Playing

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

Searching for solutions, uninformed search (BFS, DFS), heuristic search (hill climbing, A*, AO*), problem reduction, adversarial search, minimax, multiplayer games, problems in game playing, alpha-beta pruning, evaluation functions.

1. Searching for Solutions

Many AI problems (route finding, puzzles, planning) are solved by **searching**: exploring possible action sequences until a goal is found. We explore the **state space** by building a **search tree**.

- **Root node:** the initial state. **Expanding** a node means applying every legal action to generate its **child nodes**.
- **Frontier (open list):** generated nodes waiting to be expanded. **Explored set (closed list):** nodes already expanded.
- **Leaf node:** a node with no children yet. The **search strategy** decides which frontier node to expand next.

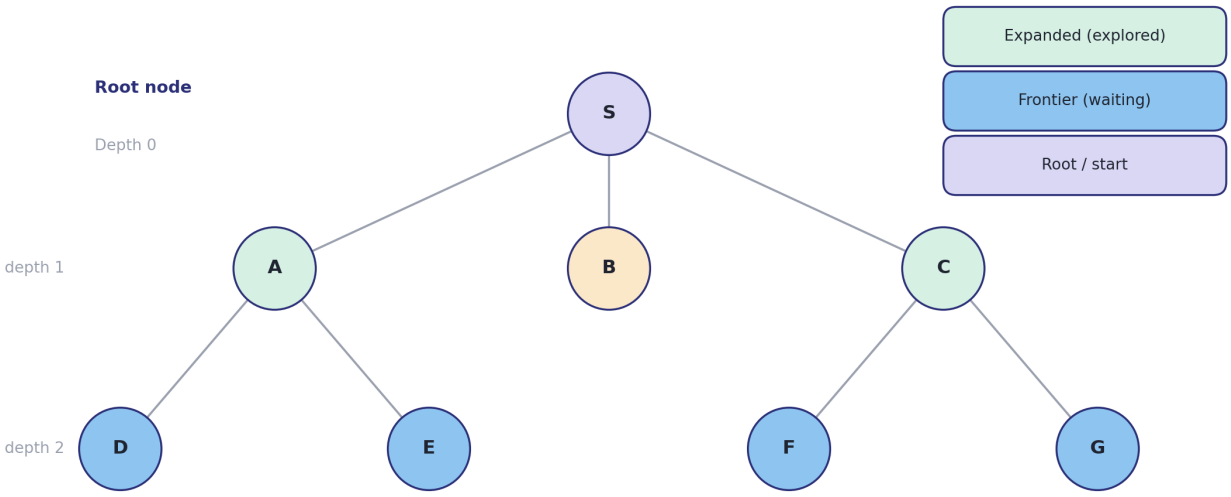


Fig 2.1 Partial search tree: expanded nodes (green), frontier (blue)

```
function TREE-SEARCH(problem) returns a solution or failure
   frontier <- { initial state of problem }
   loop do
      if frontier is empty then return failure
      node <- choose a node from frontier and remove it
      if node is a goal state then return the solution (path)
      expand node, add the resulting child nodes to frontier
```

A node in the tree stores: the **state**, the **parent** node, the **action** that created it, and the **path cost g(n)** from the root. Graph search additionally keeps an explored set to avoid repeated states (loops).

How do we compare search strategies?

Criterion	Question it answers
Completeness	If a solution exists, is the algorithm guaranteed to find it?
Optimality	Does it find the lowest-cost solution?
Time complexity	How many nodes are generated?
Space complexity	How many nodes must be stored in memory?

> **NOTE**
> Complexity is stated using three numbers: **b** = branching factor (max children per node), **d** = depth of the shallowest goal, **m** = maximum depth of the state space.

2. Uninformed (Blind) Search Strategies

Uninformed strategies have **no extra information** about how close a state is to the goal. They only know the problem definition: they can generate successors and test for a goal.

2.1 Breadth-First Search (BFS)

BFS expands the **shallowest unexpanded node** first: root, then all its children, then all grandchildren, and so on. The frontier is a **FIFO queue**.

```
BFS(problem):
   node <- (initial state, cost 0);  if goal(node) return node
   frontier <- FIFO queue with node;  explored <- empty set
   loop:
      if frontier is empty return FAILURE
      node <- POP(frontier);  add node.state to explored
      for each action in ACTIONS(node.state):
          child <- CHILD-NODE(problem, node, action)
          if child.state not in explored and not in frontier:
              if goal(child) return SOLUTION(child)
              INSERT(child, frontier)
```

2.2 Depth-First Search (DFS)

DFS expands the **deepest unexpanded node** first. It dives down one branch until a leaf is reached, then **backtracks** to the most recent node with unexplored children. The frontier is a **LIFO stack** (or recursion).

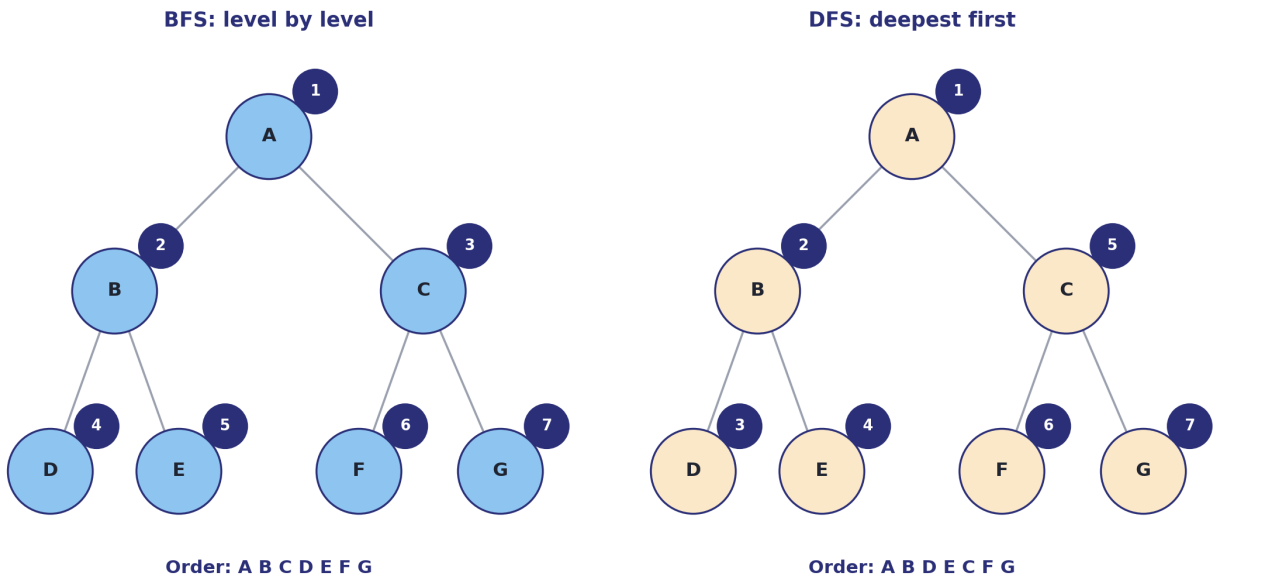


Fig 2.2 Order in which nodes are visited (small numbers) on the same tree

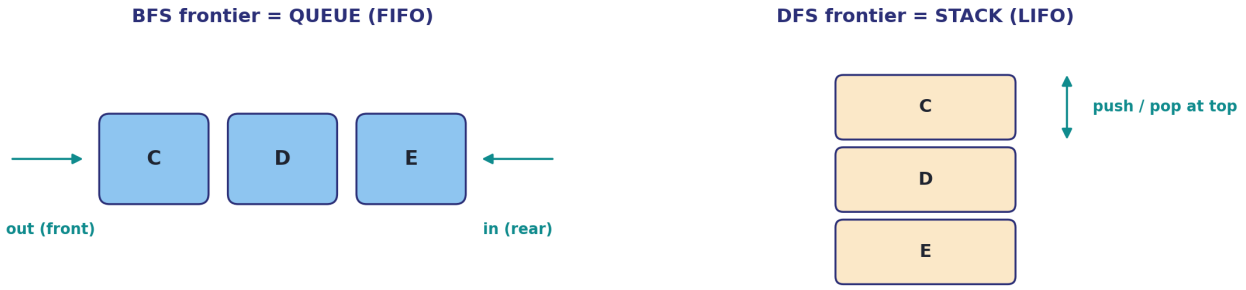


Fig 2.3 The frontier data structure decides the search order

Property	Breadth-First Search	Depth-First Search
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Complete?	Yes (if b is finite)	No (can loop or go infinitely deep); yes in finite spaces with repeat checking
Optimal?	Yes, if all step costs are equal	No
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$: the big weakness	O(b·m): linear, very small
Best used when	Shallow solution, need shortest path	Memory is limited, many solutions exist deep in the tree

EXAMPLE

With b = 10 and d = 6, BFS generates about 10^6 nodes and must keep them all in memory, which is the real bottleneck. DFS on the same tree with m = 6 stores only about b·m = 60 nodes.

Variants worth knowing: *Depth-limited search* runs DFS with a depth cut-off ℓ to avoid infinite paths. *Iterative deepening* runs depth-limited search with ℓ = 0, 1, 2, ... and combines BFS's completeness and optimality with DFS's low memory. *Uniform-cost search* expands the node with the lowest path cost g(n).

3. Search with Partial Information: Heuristic Search

Uninformed search wastes effort. **Informed (heuristic) search** uses problem-specific knowledge to head towards the goal. The knowledge is packaged as a **heuristic function h(n)**, an estimate of the cheapest cost from node n to a goal.

DEFINITION
Heuristic function h(n) = estimated cost of the cheapest path from n to the goal, with h(goal) = 0. It is a rule of thumb: not always right, but usually helpful.

Problem	Heuristic h(n)
8-puzzle	h_1 = number of misplaced tiles; h_2 = sum of Manhattan distances of tiles from their goal squares
Route finding	Straight-line distance from the current city to the destination
Chess	Material count (value of pieces) as a quick estimate of winning chances

Greedy best-first search expands the node that *looks* closest to the goal, using f(n) = h(n). It is fast but neither optimal nor complete.

3.1 Hill Climbing

Hill climbing is a **local search** method. It keeps only **one current state** and repeatedly moves to a neighbouring state with a better objective value. It stops when no neighbour is better, like climbing a mountain in thick fog by always stepping uphill. It uses very little memory and needs no search tree.

```
HILL-CLIMBING(problem):
   current <- MAKE-NODE(initial state)
   loop:
      neighbour <- the highest-valued successor of current
      if VALUE(neighbour) <= VALUE(current) then return current.state
      current <- neighbour
```

Variant	How it works
Simple hill climbing	Moves to the **first** neighbour that is better than the current state
Steepest-ascent hill climbing	Examines **all** neighbours and moves to the best one
Stochastic hill climbing	Chooses randomly among uphill moves (steeper moves are more likely)
Random-restart hill climbing	Runs hill climbing repeatedly from random starting states and keeps the best result

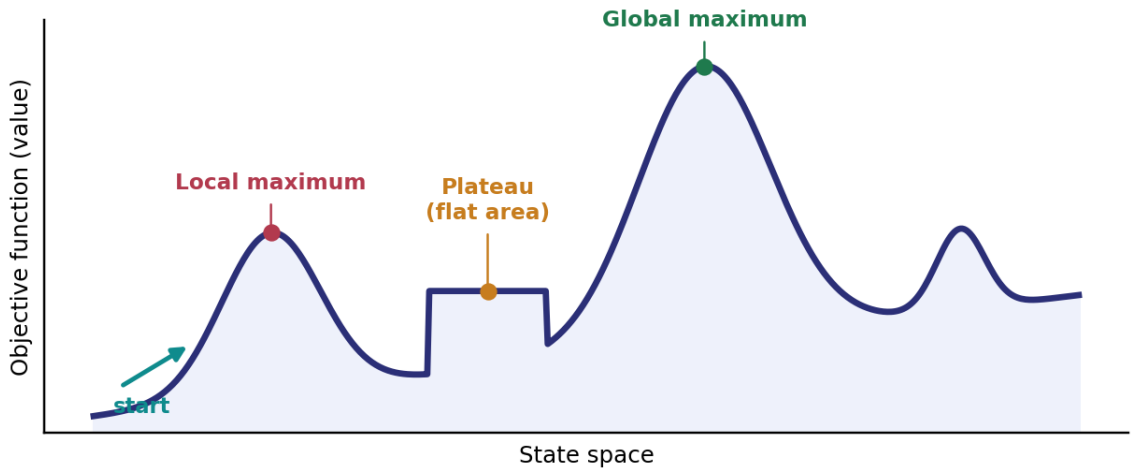


Fig 2.4 State-space landscape: where hill climbing can get stuck

Problems with hill climbing

- **Local maximum:** a peak higher than its neighbours but lower than the global maximum, so the search gets stuck.
- **Plateau:** a flat area where neighbours have equal value and there is no clear uphill direction.
- **Ridge:** a narrow slope whose peak cannot be reached by single moves, because moves go sideways instead of up.
- **Remedies:** random restarts, allowing a limited number of sideways moves, and **simulated annealing**, which occasionally accepts a downhill move to escape local peaks.

4. The A* Algorithm

A* is the most widely used informed search. It expands the node with the **lowest estimated total cost**:

> **FORMULA**
> **f(n) = g(n) + h(n)**
> g(n) = actual cost from the start to n | h(n) = estimated cost from n to the goal | f(n) = estimated cost of the cheapest solution *through* n

```
A*(start, goal):
   OPEN <- {start};  CLOSED <- {};   g(start) = 0;  f(start) = h(start)
   while OPEN is not empty:
       n <- node in OPEN with the lowest f(n)
       if n is the goal: return the path (follow parent pointers)
       move n from OPEN to CLOSED
       for each successor m of n:
           new_g <- g(n) + cost(n, m)
           if m is new, or new_g < g(m):
               g(m) <- new_g;  f(m) <- g(m) + h(m);  parent(m) <- n
               put m in OPEN (reopen it if it was in CLOSED)
   return FAILURE
```

Worked example. Find the cheapest path from S to G in the graph below (edge costs and h-values are shown).

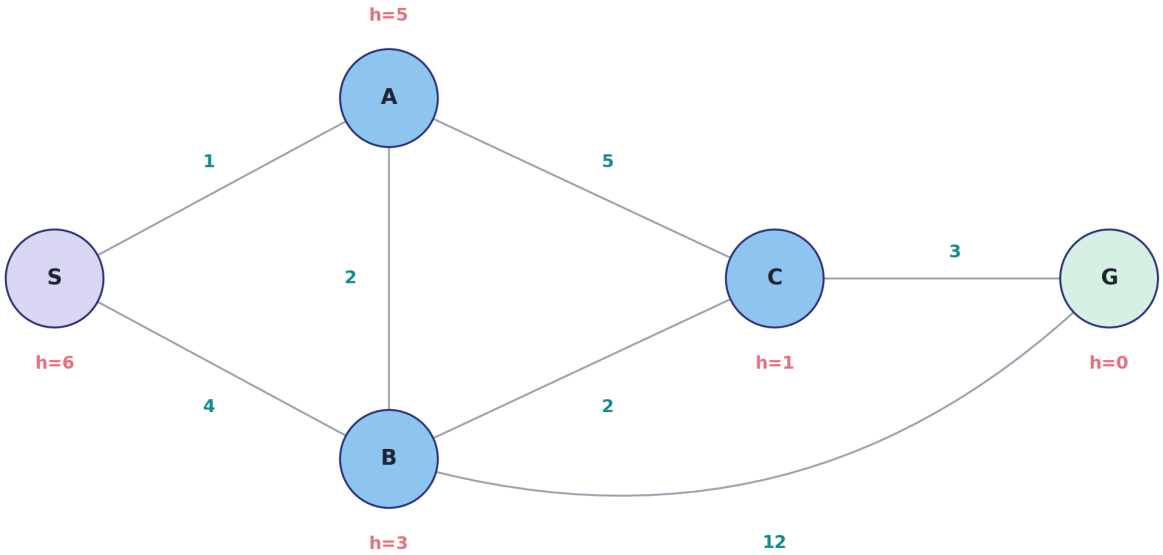


Fig 2.5 Weighted graph for A (edge labels = step cost g; red h = heuristic estimate to G)*

Step	Node expanded	Generated / updated nodes: g, f = g + h	OPEN list after the step (lowest f first)
1	S (f = 6)	A: g=1, f=6; B: g=4, f=7	A(6), B(7)
2	A (f = 6)	B improves: g=3, f=6; C: g=6, f=7	B(6), C(7)
3	B (f = 6)	C improves: g=5, f=6; G: g=15, f=15	C(6), G(15)
4	C (f = 6)	G improves: g=8, f=8	G(8)
5	G	Goal reached	Done

RESULT
Optimal path: **S → A → B → C → G** with cost 1 + 2 + 2 + 3 = **8**. Note how A* ignored the direct but expensive edge B → G (cost 12).

Properties of A*

- **Admissible heuristic:** h(n) never *over*estimates the true cost, h(n) ≤ h*(n). With an admissible h, tree-search A* is **optimal**.
- **Consistent (monotonic) heuristic:** h(n) ≤ c(n, a, n') + h(n'). Consistency implies admissibility, and f never decreases along a path.
- **Complete** (if b is finite and step costs are positive), and **optimally efficient** for a given heuristic.
- **Weakness:** keeps all generated nodes in memory, so space is the main limit, $O(b^d)$ in the worst case.

EXAM TIP
Special cases: if h(n) = 0 for all n, A* becomes **uniform-cost search**. If g(n) is ignored it becomes **greedy best-first search**.

5. Problem Reduction and the AO* Algorithm

Some problems break naturally into **independent sub-problems**. Solving the whole problem means solving *all* of them. This is **problem reduction**: the problem is repeatedly reduced to easier sub-problems, until only primitive (directly solvable) problems remain.

Such problems are represented by an **AND-OR graph** (or tree):

- **OR arcs:** alternative ways to solve a problem. *Any one* branch is enough (like ordinary search).
- **AND arcs:** sub-problems that must *all* be solved. A curved arc joins the AND branches.
- A **solution** is not a path but a **sub-graph** that solves every node reached through AND arcs.

EXAMPLE
Goal: *Get a degree*. OR-ways: (1) **buy a certificate** (single step) or (2) AND-way: **pass all semester exams AND complete the project**. Option 2 needs both parts to succeed.

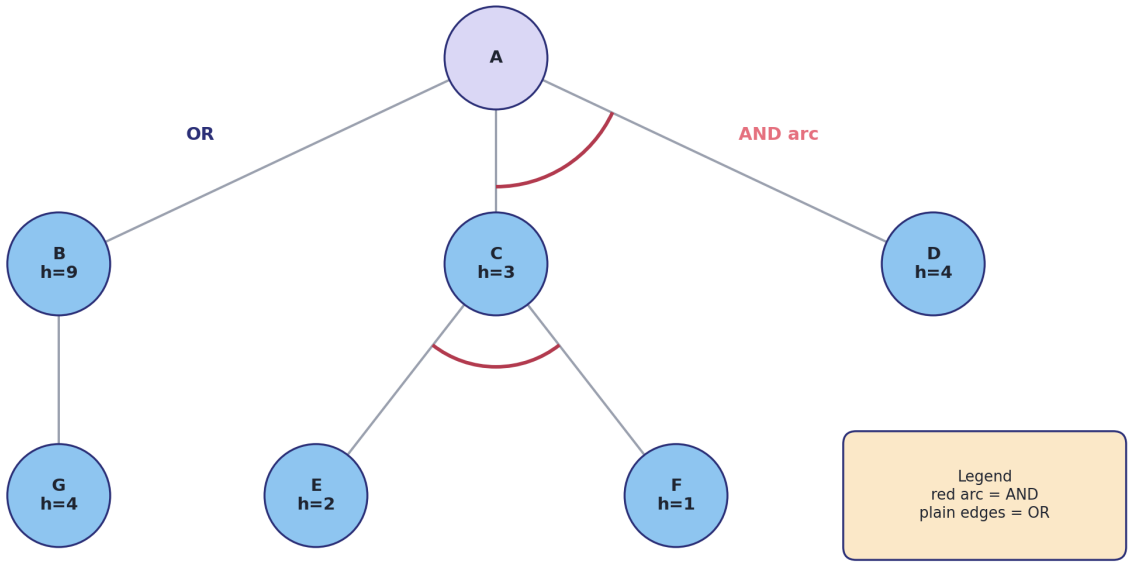


Fig 2.6 AND-OR graph for AO (A: OR of B, or AND of C and D; every edge costs 1)*

A* cannot handle AND-OR graphs, because the cost of an AND arc depends on **several** children. **AO*** is the heuristic algorithm for AND-OR graphs. For a node n and each arc to child set $\{n_1 \ldots n_k\}$, the cost of that arc is **Σ ($h(n_i)$ + cost of the edge to n_i)**. The node takes the **minimum** over its arcs.

AO* algorithm (steps)

- **1.** Start with the initial node as the current best partial solution graph; give it its heuristic value.
- **2.** Pick a non-terminal leaf node of the current **best** partial solution graph and **expand** it.
- **3.** For each new successor set, compute its cost = sum of (h + edge cost) for an AND arc, or h + edge cost for an OR arc.
- **4. Back-propagate (revise) costs** up to the root: each ancestor takes the cheapest arc. Mark the best arcs.
- **5.** Repeat 2-4 until the root is marked **SOLVED** (all nodes on the best graph are solved) or its cost exceeds a limit.

Worked example (graph of Fig 2.6)

Step	What we do	Cost computed	Best arc for A
1	Expand A: OR arc to B; AND arc {C, D}	via B = 9 + 1 = 10 via C&D = (3+1) + (4+1) = 9	AND {C, D} (9)
2	Expand C into AND {E, F}	f(C) = (2+1) + (1+1) = 5 (was 3). Revised: via C&D = (5+1) + (4+1) = 11	Now B is cheaper (10 < 11): **switch to B**
3	Expand B into G	f(B) = 4 + 1 = 5 (was 9). via B = 5 + 1 = 6	B (6): **best, solved**

Notice how AO* **revises its choice** when better information appears; this cost back-propagation is the key idea that A* does not have.

6. Game Playing: Adversarial Search

In **multi-agent competitive environments** the goals of agents conflict: one agent's gain is another's loss. **Adversarial search** decides moves in such **games** (chess, checkers, tic-tac-toe, Go). Games are interesting for AI because they are hard (huge search spaces), have clear rules and a measurable outcome.

	Deterministic	Chance involved
Perfect information	Chess, Checkers, Go, Tic-Tac-Toe	Backgammon, Monopoly
Imperfect information	Battleship, Kriegspiel	Bridge, Poker, Scrabble

A game as a search problem

Element	Meaning
S_0 (initial state)	How the game is set up at the start
PLAYER(s)	Which player has the move in state s
ACTIONS(s)	Set of legal moves in s
RESULT(s, a)	Transition model: the state after move a in state s
TERMINAL-TEST(s)	True when the game is over
UTILITY(s, p)	Final numeric value for player p at terminal state s (e.g., +1 win, 0 draw, −1 loss)

The **game tree** has states as nodes and moves as edges. We call the two players **MAX** (tries to maximise utility) and **MIN** (tries to minimise it). Each pair of moves (one by MAX and one by MIN) is called a **ply** pair; tic-tac-toe has at most 9 plies.

6.1 Minimax algorithm

Minimax computes the **optimal decision** for MAX, assuming MIN also plays perfectly. Every node gets a **minimax value**:

RULE
MINIMAX(s) = UTILITY(s) if s is terminal
= **max** of MINIMAX(RESULT(s, a)) if PLAYER(s) = MAX
= **min** of MINIMAX(RESULT(s, a)) if PLAYER(s) = MIN

```
function MINIMAX-DECISION(state) returns an action
   return argmax over a in ACTIONS(state) of MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state):  if TERMINAL(state) return UTILITY(state)
   v <- -infinity;  for each a: v <- MAX(v, MIN-VALUE(RESULT(state, a)));  return v
function MIN-VALUE(state):  if TERMINAL(state) return UTILITY(state)
   v <- +infinity;  for each a: v <- MIN(v, MAX-VALUE(RESULT(state, a)));  return v
```

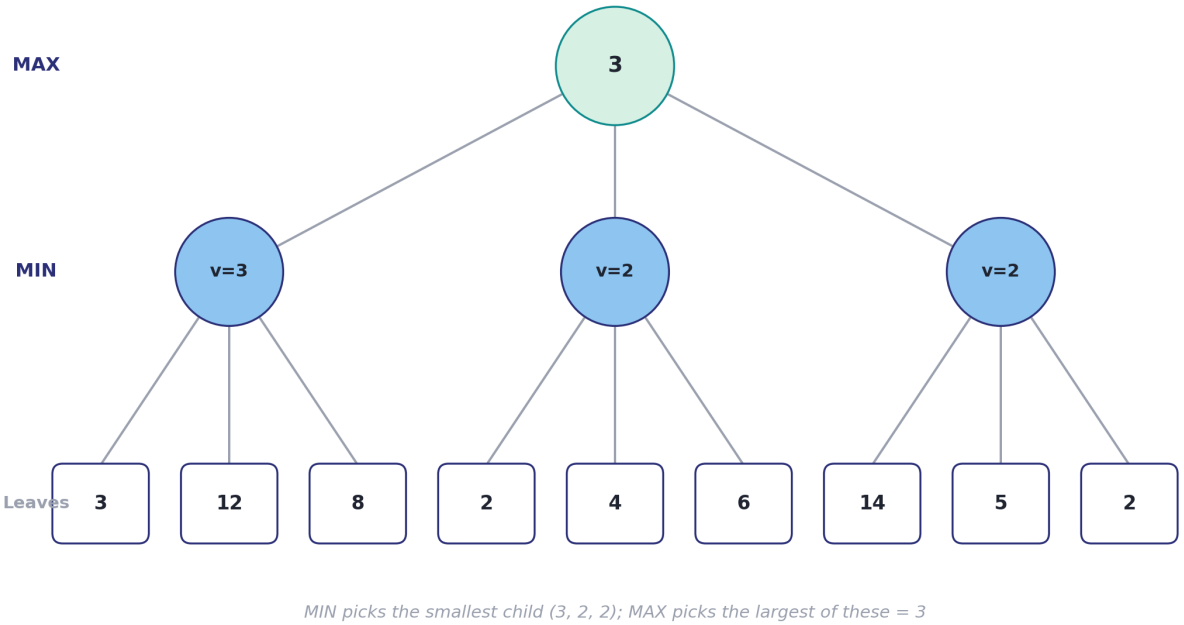


Fig 2.7 Minimax on a two-ply game tree

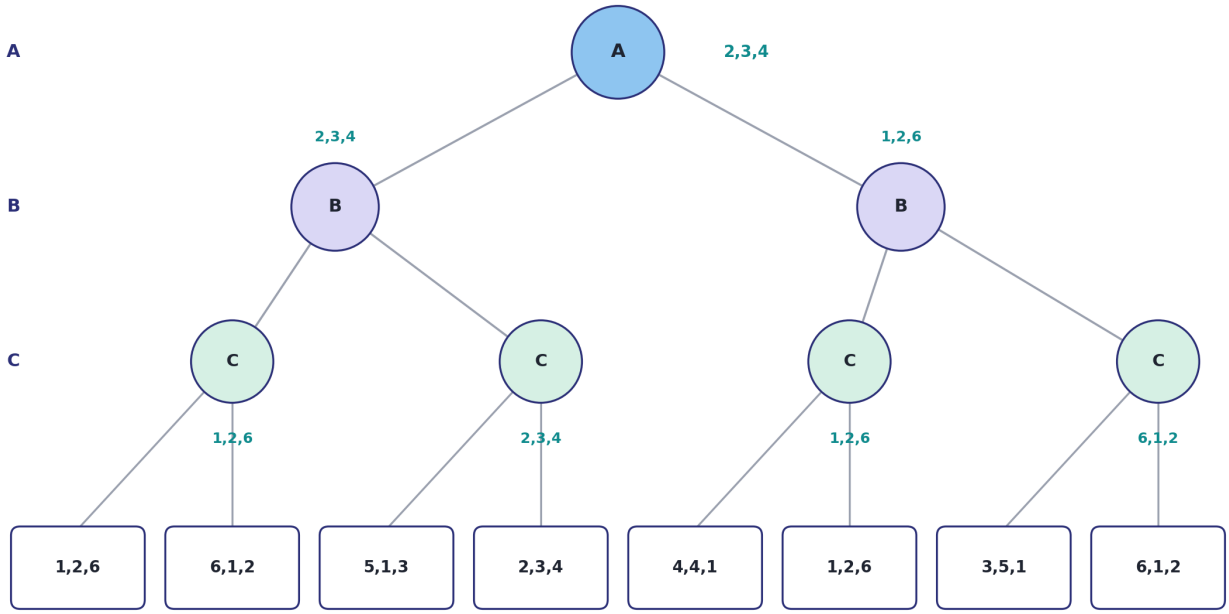
Reading the tree: the three MIN nodes pick the smallest of their leaves: min(3,12,8) = 3, min(2,4,6) = 2, min(14,5,2) = 2. The MAX root then picks the largest of {3, 2, 2} = **3**, so MAX should play the left move.

Property	Minimax
Complete	Yes, if the game tree is finite
Optimal	Yes, against an optimal opponent
Time	$O(b^m)$
Space	O(b·m) with depth-first exploration

> **NOTE**
> For chess b ≈ 35 and m ≈ 100, so b^m is about 10^{154} nodes. Exact minimax is impractical, which motivates pruning and evaluation functions.

6.2 Optimal decisions in multiplayer games

With more than two players, each node stores a **vector** of values, one utility per player. Each player, when it is their turn, chooses the move that maximises **their own component** of the vector.



Leaf = (v_A, v_B, v_C). Each player picks the child with the highest value in THEIR OWN slot.

Fig 2.9 Three-player game: each node holds a vector of utilities, one per player

- **Alliances** can form: if A and B are both weak against C, cooperating against C may be rational for both of them.
- In pure 2-player zero-sum games, the vector collapses to a single number and the vector method becomes standard minimax.
- Multiplayer games are usually **non-zero-sum**, so cooperation can emerge naturally.

7. Problems in Game Playing

- **Huge search space:** the game tree grows exponentially, so the whole tree can never be searched. We must stop at a depth limit and estimate.
- **Horizon effect:** the search stops at a fixed depth and may miss a disaster just beyond the horizon, or postpone it with pointless delaying moves.
- **Quiescence problem:** evaluating a position in the middle of an exchange (e.g., a capture) gives a misleading value. Remedy: **quiescence search**, which extends the search until the position is stable.
- **Imperfect evaluation:** the evaluation function is only a guess, and its errors can accumulate.
- **Opponent modelling and uncertainty:** the opponent may not play optimally; chance elements (dice) need **expectiminimax**.
- **Time limits:** real games have clocks, so we need **iterative deepening** and good move ordering.

8. Alpha-Beta Pruning

Minimax must examine every node. **Alpha-beta pruning** returns the **same result** as minimax but **skips branches** that cannot influence the final decision. It keeps two bounds:

- **α (alpha)** = the best (highest) value found so far for MAX along the path. It starts at $-\infty$.
- **β (beta)** = the best (lowest) value found so far for MIN along the path. It starts at $+\infty$.
- **Prune** (stop exploring a node's remaining children) whenever **α ≥ β**.

```
function ALPHA-BETA-SEARCH(state): v <- MAX-VALUE(state, -inf, +inf); return the action with value v

function MAX-VALUE(state, alpha, beta):
   if TERMINAL(state) return UTILITY(state)
   v <- -inf
   for each a in ACTIONS(state):
        v <- MAX(v, MIN-VALUE(RESULT(state, a), alpha, beta))
        if v >= beta then return v          # beta cut-off
        alpha <- MAX(alpha, v)
   return v
function MIN-VALUE(state, alpha, beta):     # symmetric
   v <- +inf
   for each a in ACTIONS(state):
        v <- MIN(v, MAX-VALUE(RESULT(state, a), alpha, beta))
        if v <= alpha then return v         # alpha cut-off
        beta <- MIN(beta, v)
   return v
```

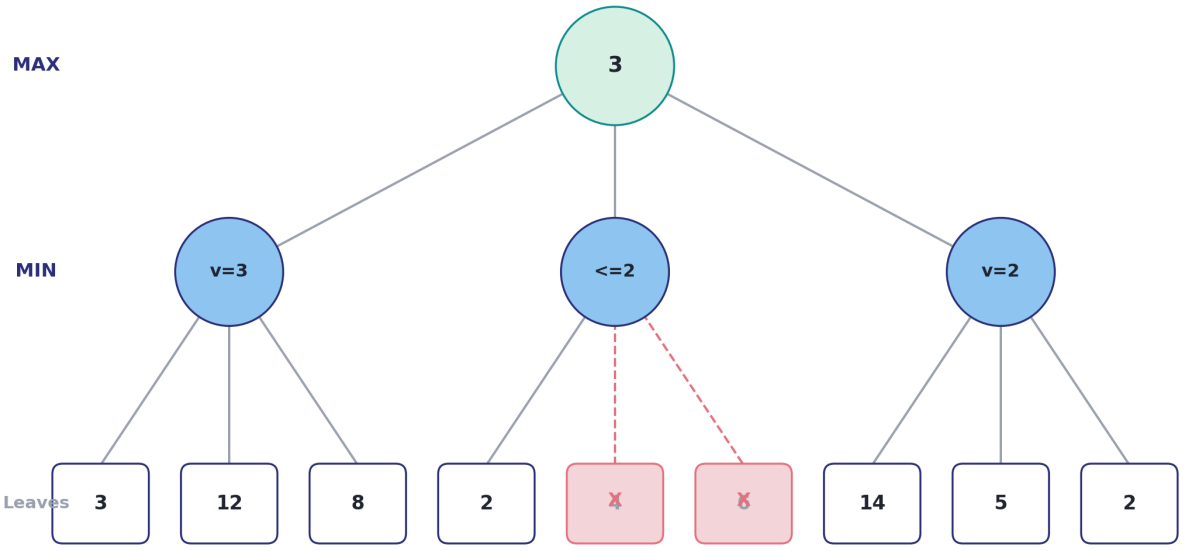


Fig 2.8 Alpha-beta pruning on the same tree

Trace: At the left MIN node, v becomes 3, so α = 3 at the root. At the middle MIN node the first leaf is 2, so v ≤ 2 ≤ α = 3: MAX will never choose this branch, so the leaves **4 and 6 are pruned**. The right node is searched (14, 5, 2) and gives 2. The root value is still **3**, the same as minimax, with fewer nodes examined.

Aspect	Details
Correctness	Pruning never changes the final minimax value or the best move
Best case (perfect move ordering)	Time $O(b^{m/2})$: searches **twice as deep** in the same time. Branching factor effectively becomes √b
Worst case	No pruning, $O(b^m)$, same as minimax
Move ordering	Try the best moves first (e.g., captures first in chess) to prune more; use iterative deepening to learn good orderings

Evaluation functions

Because we cannot search to the end of the game, we **cut off** the search at some depth and apply a **heuristic evaluation function EVAL(s)** that estimates the expected utility of the state, as if it were a terminal state.

FORMULA
EVAL(s) = $w_1f_1(s) + w_2f_2(s) + \ldots + w_nf_n(s)$ (weighted linear function of features)

- **Features f_i** are measurable properties of the position; **weights w_i** show their importance (these can be learned).
- **Chess example:** f_1 = (white pawns − black pawns) with weight 1, f_2 = (white knights − black knights) with weight 3, bishops 3, rooks 5, queens 9.
- **Requirements of a good EVAL:** it must agree with the true utility at terminal states, be **fast to compute**, and correlate strongly with the actual chance of winning.
- **Cut-off test:** replaces the terminal test; for example, stop at depth d, or when the position is quiet (quiescence).

Quick Revision: Unit II at a Glance

Topic	One-line recall
BFS / DFS	Queue, shallow-first, complete and optimal (equal costs) / stack, deep-first, low memory, not optimal
Heuristic h(n)	Estimate of the remaining cost to the goal; admissible = never overestimates
Hill climbing	Always move uphill; may get stuck at a local maximum, plateau or ridge
A*	f(n) = g(n) + h(n); optimal with an admissible heuristic
AO*	Heuristic search on AND-OR graphs; sum costs on AND arcs and revise costs upward
Minimax	MAX picks the highest and MIN the lowest value; assumes optimal play
Alpha-beta	Same answer as minimax, prune when $\alpha \geq \beta$; best case $O(b^{m/2})$
Evaluation function	Weighted sum of features used when the search is cut off

Frequently asked exam questions

- Explain BFS and DFS with algorithms, examples and a comparison of their properties.
- What is a heuristic? Explain hill climbing and its problems. How can they be overcome?
- Explain the A* algorithm with an example. When is A* optimal?
- Explain problem reduction and the AO* algorithm with an AND-OR graph.
- Explain the minimax algorithm. Write about optimal decisions in multiplayer games.
- Explain alpha-beta pruning with an example. What is an evaluation function?